# Technical Design Document

AI Schema Builder

Technical Design Document (TDD)

## Version 1.0 | 2025

## 1. System Design

## 1.1 Database Schema (Application DB)

The application stores user data in PostgreSQL. Core tables are described below.

## Table

## Purpose

## Key Columns

users

Registered users

id, email, password_hash, created_at

projects

User projects

id, user_id, name, rdbms_type, created_at

tables

Tables within a project

id, project_id, name, description

schemas

Generated schema for a table

id, table_id, version, columns_json, created_at

references

FK relationships between tables

id, project_id, from_table_id, from_col, to_table_id, to_col, accepted

api_keys

Encrypted Google Gemini API keys

id, user_id, encrypted_key, iv, created_at

## 1.2 Gemini API Integration

Free API key: https://aistudio.google.com — no billing or credit card required. Model: gemini-2.5-flash. Free tier gives 250 requests/day and 10 RPM.

Sample Node.js backend call for schema generation:

```
const GEMINI_URL =

'https://generativelanguage.googleapis.com/v1beta/models/

gemini-2.5-flash:generateContent?key=' + process.env.GEMINI_API_KEY;

const prompt = [

'You are a database architect. Return ONLY valid JSON. No markdown.',

'Project: ' + project.name + ' | RDBMS: ' + project.rdbms,

'Existing tables: ' + existingTables.join(', '),

'Table: ' + table.name + ' — ' + table.description,

'Format: {columns:[{name,type,nullable,primaryKey,foreignKey?}],indexes:[]}'

].join('

');

const res = await fetch(GEMINI_URL, {

method: 'POST',

headers: { 'Content-Type': 'application/json' },

body: JSON.stringify({

contents: [{ parts: [{ text: prompt }] }],

generationConfig: { responseMimeType: 'application/json' }

})

});

const data = await res.json();

const schema = JSON.parse(data.candidates[0].content.parts[0].text);
```

# 2. API Specification

## 2.1 Schema Generation Endpoint

```
POST /api/schemas/generate
```

## Field

## Type

## Required

## Description

projectId

UUID

## Yes

Project to generate schema within

tableId

UUID

## Yes

Target table for schema generation

model

string

## No

Gemini model override (default: gemini-2.5-flash)

Response: 200 OK

{

"schemaId": "uuid",

"version": 1,

"columns": [...],

"indexes": [...],

"tokensUsed": 842

}

## 2.2 Reference Detection Endpoint

POST /api/schemas/references — Analyzes all accepted schemas in a project and returns suggested FK relationships.

## Field

## Type

## Description

projectId

UUID

Project to analyze

forceRegenerate

boolean

Re-analyze even if refs already exist

## 3. Security Design

## 3.1 API Key Encryption

User-provided Gemini API keys are encrypted using AES-256-GCM before storage. The encryption key is stored in AWS Secrets Manager and never in the application codebase.

```
const crypto = require('crypto');

const ALGO = 'aes-256-gcm';

function encryptKey(plaintext, masterKey) {

const iv = crypto.randomBytes(16);

const cipher = crypto.createCipheriv(ALGO, masterKey, iv);

const encrypted = Buffer.concat([

cipher.update(plaintext, 'utf8'),

cipher.final()

]);

const tag = cipher.getAuthTag();

return { iv: iv.toString('hex'),

data: encrypted.toString('hex'),

tag: tag.toString('hex') };

}
```

## 3.2 Authentication Flow

User registers → bcrypt hashes password (12 rounds) → stored in DB

User logs in → compare hash → issue JWT (24h expiry) + refresh token (7d)

API calls include Authorization: Bearer <token> header

Backend middleware verifies JWT on every protected route

Refresh tokens are rotated on each use (sliding window)

## 4. Frontend Component Architecture

## Component

## Responsibility

ProjectList

Display all user projects with quick actions

TableManager

Add/edit/delete tables within a project

SchemaEditor

Editable grid for reviewing/modifying AI-generated schema

ERDViewer

React Flow canvas rendering table nodes and FK edges

GenerateButton

Triggers Gemini schema generation with loading state

ExportModal

Choose export format (SQL/JSON/Markdown) and download

APIKeySettings

Manage Google Gemini API key (input, save, verify, delete)

ReferenceReviewer

Accept/reject FK reference suggestions from Gemini

## 5. Error Handling & Resilience

## 5.1 Gemini API Errors

## Error

HTTP Status

## Handling Strategy

Rate limit (429)

## 429

Exponential backoff: 1s, 2s, 4s, 8s — max 3 retries

## Overloaded (529)

## 529

Retry with 5s delay up to 2 times

Invalid JSON response

## 200

Parse error caught — return 422 to user with prompt to retry

API key invalid (401)

## 401

Return 400 to user: 'Invalid Google Gemini API key'

Timeout (> 30s)

## N/A

Cancel request, return 504 with retry suggestion

## 5.2 Input Validation

All API inputs validated with Zod schemas before processing

Table names validated against SQL reserved words and injection patterns

Maximum table description length: 1000 characters

Maximum 200 tables per project enforced at API level

End of Technical Design Document